# 📄 ExoHabitAI – Frontend Implementation Documentation

---

# 1. Introduction

The frontend of ExoHabitAI is designed to provide an intuitive and visually appealing interface for users to input planetary parameters and receive habitability predictions.

The UI communicates with a Flask backend API that processes the data using a trained XGBoost machine learning model.

The frontend is built using:

- HTML5
- CSS3
- Bootstrap 5
- JavaScript (Fetch API)

# 2. Frontend Architecture

The frontend is integrated directly with the Flask backend using:

- **templates/** folder for HTML
- **static/** folder for CSS and JavaScript

### Final Folder Structure

backend/
│

```
├── templates/
│   └── index.html
│
└── static/
    ├── style.css
    └── script.js
```

Flask renders the frontend using:

return render_template("index.html")

# 3. User Interface Design

## 3.1 Theme & Visual Design

The application uses a **Dark Blue Gradient Theme** for a modern scientific look.

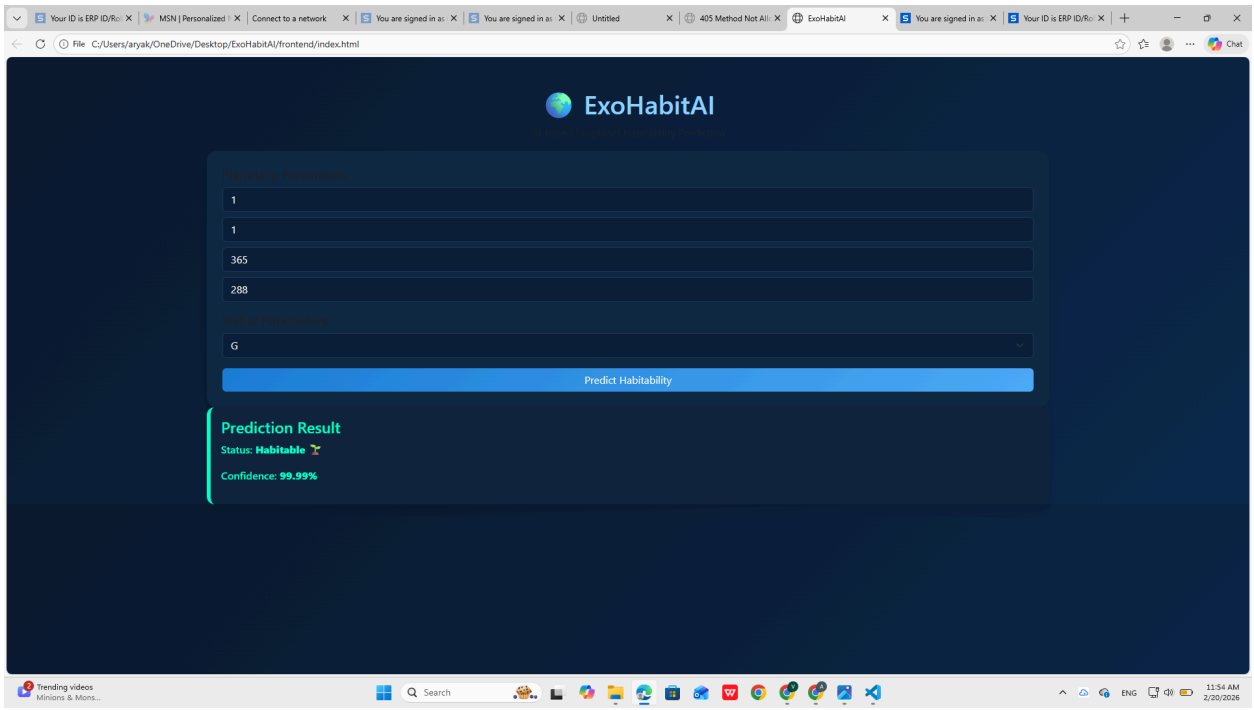
### Design Features:

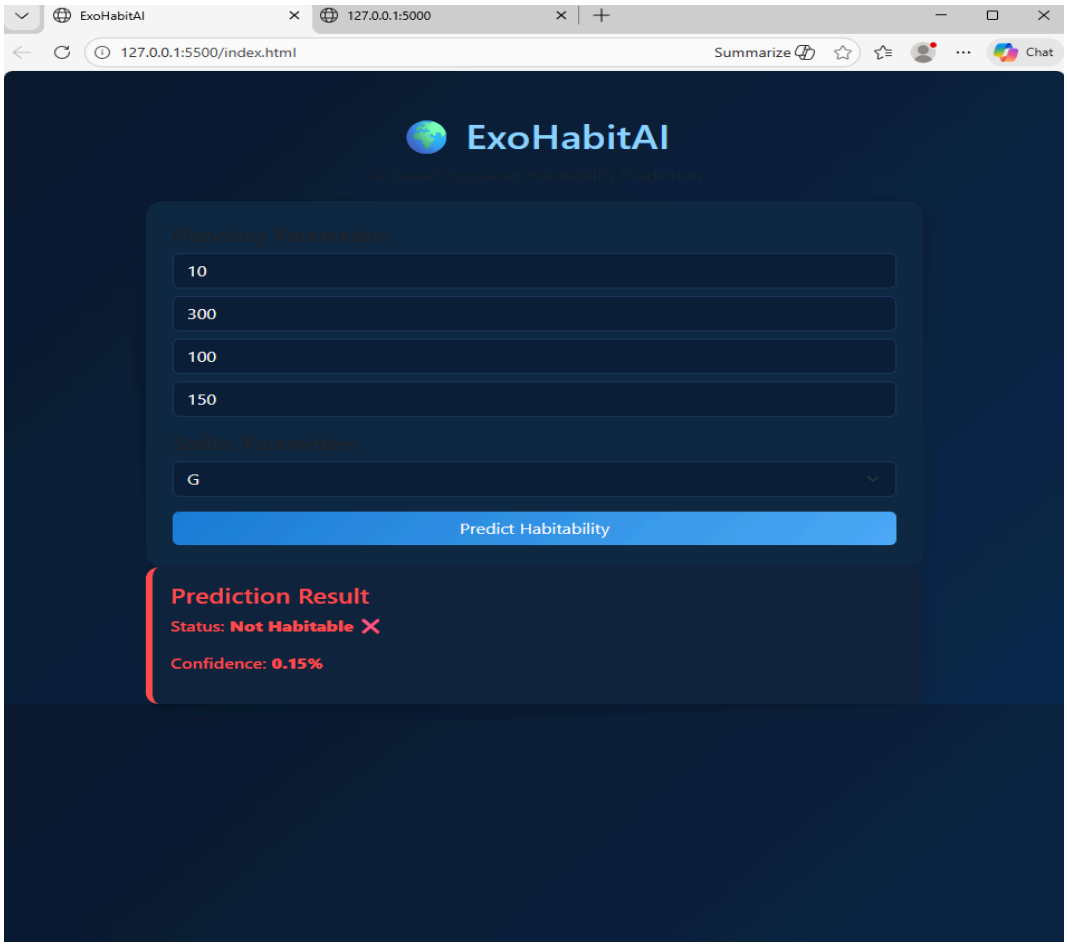
- Background: Linear gradient (Deep Blue tones)
- Card-based layout
- Soft rounded corners
- Clean typography (Segoe UI)
- Bootstrap 5 styling

### Color Scheme:

- Primary Blue: #4dabf7
- Dark Navy Background: #0a1a2f
- Success (Habitable): Green highlight
- Error (Not Habitable): Red highlight

## 3.2 User Interface Screenshot






# 4. Input Fields

The frontend collects the following parameters:

| Parameter | Description |
|---|---|
| Planet Radius | In Earth radii |
| Planet Mass | In Earth masses |
| Orbital Period | In days |
| Equilibrium Temperature | In Kelvin |
| Star Spectral Type | M, K, G, F, A |

Each field includes:

- Proper labels
- Numeric validation
- Dropdown selection for star type

# 5. Form Handling & API Integration

The form submission is handled using JavaScript.

### Key Features:

- Prevents default page reload
- Converts inputs to float values
- Sends JSON data to backend
- Displays loading message
- Handles API errors gracefully

### Fetch API Request:

```
fetch("/predict", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data)
})
```

# 6. Prediction Display Logic

After receiving backend response:

{

"prediction": 1,
"confidence": 99.87
}

The UI dynamically updates the result section.

### Display Features:

- Prediction Result Card
- Habitability Status:
    - 🌱 Habitable (Green)
    - ❌ Not Habitable (Red)
- Confidence Score Percentage

### Dynamic Styling:

JavaScript adds class:

resultDiv.className = isHabitable ? "habitable" : "not-habitable";

This changes the visual appearance accordingly.

# 7. Error Handling

The frontend handles:

- Invalid input errors
- API response errors
- Network errors
- Backend validation errors

If error occurs:

```
<div class="alert alert-danger">
  Error Message
</div>
```

# 8. Responsiveness

The UI is fully responsive using:

- Bootstrap Grid System
- Mobile-friendly layout
- Adaptive input spacing
- Card-based alignment

Tested on:

- Desktop browsers
- Laptop screens
- Mobile view (Chrome DevTools)

# 9. User Experience Enhancements

- Button disables during prediction
- Loading message displayed
- Clean visual feedback
- Clear confidence display
- Modern scientific aesthetic

# 10. Deployment Integration

After deployment on Render:

- Frontend is served via Flask
- Static files loaded via:

```
{{ url_for('static', filename='style.css') }}
{{ url_for('static', filename='script.js') }}
```

- Fetch API uses relative path:

```
fetch("/predict")
```

Ensuring compatibility in production.

# 11. Testing Scenarios

The frontend was tested using:

- Valid habitable values
- Non-habitable extreme values
- Edge cases
- Invalid numeric inputs
- Empty field validation

All scenarios returned appropriate responses.

# 12. Conclusion

The ExoHabitAI frontend successfully:

- Provides a clean and modern UI
- Integrates seamlessly with Flask backend
- Displays real-time ML predictions
- Handles errors gracefully
- Works in production environment

The frontend ensures a smooth and intuitive user experience while maintaining professional design standards suitable for scientific applications.